Database II Lab/ 3rd Grade

[Second Semester] and [2026]

[Lab 5] [17/3/ 2026] [8.30 + 10.30 am]

Instructor Information

Instructor

Dr. Rasool Hisham [rasool.hisham@nahrainuniv.edu.iq] [2 Hrs]

Dr. Zainab Namh

Dr. Azhar Flaih

Assist. Prof. Zahraa Jaaz

Email Hours

LAB 5 — CRUD OPERATIONS ON MYSQL USING C# WINDOWS FORMS

Lab Objectives

By the end of this lab, students will be able to:

- Perform CRUD operations (Create, Read, Update, Delete) using ExecuteNonQuery().
- Write secure parameterized SQL queries to prevent SQL Injection attacks. • Handle user interaction using the MessageBox API.

The ExecuteNonQuery() Method

In C# with MySQL, ExecuteNonQuery() is a method of the MySqlCommand class that executes SQL statements which do NOT return rows of data. It is the primary method for all write operations on a database.

When to Use ExecuteNonQuery()

SQL Category	SQL Statements	Returns
DML — Data Manipulation	INSERT, UPDATE, DELETE	Number of rows affected (int)
DDL — Schema Changes	CREATE TABLE, ALTER TABLE, DROP TABLE	0
DCL — Control	GRANT, REVOKE	0

```
// Returns the number of rows the SQL statement changed
int rowsAffected = cmd.ExecuteNonQuery();

if (rowsAffected > 0)
 MessageBox.Show("Success! " + rowsAffected + " row(s) changed.");
else
 MessageBox.Show("No rows were changed.");
```

Establishing a Database Connection

Every database operation in C# follows the same lifecycle: Open → Execute → Close. The using block is the recommended pattern because it guarantees the connection is closed even if an exception occurs.

Connection String Components

Component	Purpose	Example Value
server	Hostname or IP of the MySQL server.	Localhost or lab3a.ncsd.edu
user	MySQL username.	root or u1
password	MySQL user password.	12345
database	Name of the database to connect to.	company
port	Port number (default: 3306).	3306

Safe Connection with the using Block

The using keyword in C# creates a scope for any object that implements IDisposable. When the block ends (normally or due to an exception), .Dispose() is called automatically, which internally calls .Close() on a database connection.

```
string connString =
"server=localhost;user=root;password=1234;database=company;";

// The 'using' block guarantees the connection is ALWAYS
closed. using (var conn = new MySqlConnection(connString))
{
 conn.Open();

 // All database work goes here

} // <- conn.Dispose() (which conn.Close()) is called automatically here
```

⚠️ Common Mistake — Not Closing Connections

If you call conn.Close() manually and an exception is thrown BEFORE that line, the connection stays open.

Open connections consume server memory and can exhaust the connection pool. Solution: ALWAYS use the 'using' block — it handles closure for you.

CRUD Operations Step by Step

This section covers all four CRUD operations on the department table of the company database. Each operation follows the same structural pattern.

CREATE — Inserting a New Record

The INSERT statement adds a new row to a table.

1. Define the connection string with your database credentials.
2. Create a MySqlConnection object using the connection string.
3. Open the connection inside a using block.
4. Write the INSERT SQL statement.
5. Create a MySqlCommand object, passing the SQL and connection.
6. Execute the command using ExecuteNonQuery().
7. Wrap everything in try-catch to handle errors gracefully.

```
string connString =
"server=localhost;user=root;password=1234;database=company;";

try
{
 using (var conn = new MySqlConnection(connString))
 {
 conn.Open();

 // Step 4: Define the SQL
 string sql = "INSERT INTO department(dno, dname) VALUES (6, 'Test
Dept')";

 // Step 5: Create the command
 var cmd = new MySqlCommand(sql, conn);

 // Step 6: Execute
 int rows = cmd.ExecuteNonQuery();
 MessageBox.Show(rows + " row(s) inserted!", "Success",
MessageBoxButtons.OK, MessageBoxIcon.Information);  }
}
catch (Exception ex)
{
 MessageBox.Show("Error: " + ex.Message, "Database Error",
MessageBoxButtons.OK, MessageBoxIcon.Error); }
```


►► Primary Key Violation

If you try to INSERT a record with a dno that already exists, MySQL throws a duplicate key error.

The catch block will intercept this and display the error message via MessageBox.Show(ex.Message).

UPDATE — Modifying an Existing Record

The UPDATE statement modifies existing rows. Always use a WHERE clause to target specific records — without it, ALL rows in the table will be updated.

```
string sql = "UPDATE department SET dname = 'HR Division' WHERE dno = 6";

using (var conn = new MySqlConnection(connString))
{
 conn.Open();
 var cmd = new MySqlCommand(sql, conn);
 int rows = cmd.ExecuteNonQuery();
 MessageBox.Show(rows + " row(s) updated!");
}
```

⚠️ Missing WHERE Clause

UPDATE department SET dname = 'HR Division'; // NO WHERE clause The statement above will update EVERY row in the department table!
Always double-check that your WHERE condition correctly targets only the intended rows.

DELETE — Removing a Record

The DELETE statement removes rows from a table. As with UPDATE, always specify a WHERE clause.

```
string sql = "DELETE FROM department WHERE dno = 6";

using (var conn = new MySqlConnection(connString))
{
 conn.Open();
 var cmd = new MySqlCommand(sql, conn);
 int rows = cmd.ExecuteNonQuery();
 MessageBox.Show(rows > 0 ? "Deleted!" : "Record not found."); }
```


SQL Injection & Parameterized Queries

What is SQL Injection?

SQL Injection is a cyberattack technique where a malicious user inserts SQL code into an input field with the intent of manipulating the database query.

Consider a login form where the application builds its SQL like this:

```
// DANGEROUS — never do this!
string username = txtUsername.Text; // attacker types: admin' --
string sql = "SELECT * FROM users WHERE username = '" + username +
"'";

// The resulting SQL becomes:
// SELECT * FROM users WHERE username = 'admin' --'
// The -- comments out the rest, bypassing the password check entirely!
```

⚠⚠ Real-World Impact of SQL Injection

An attacker can retrieve all user records, passwords, and credit card data. They can delete entire tables with: '; DROP TABLE users; --
They can bypass authentication without knowing any password.
Parameterized queries are the PRIMARY defence — always use them.

Parameterized Queries — The Safe Solution

A parameterized query separates the SQL structure from the data values. The database driver sends them to the server separately, so the server always treats the data as literal data — never as executable code. **Parameterized Queries** is the most effective method to prevent **SQL injection**. It separates the SQL code from the data, ensuring that user input is treated strictly as data, not executable code.

How Parameterized Queries Work

1. Write SQL with named placeholders using the @ symbol (e.g., @dno, @dname). 2. Create the MySqlCommand object with the SQL template.
3. Bind actual values to each placeholder using cmd.Parameters.AddWithValue().
4. The MySQL driver sends the SQL structure and data SEPARATELY to the server. 5. The server ALWAYS treats the data as data — never as SQL code.

Safe DELETE using a parameterized query:

```
int dno = 6; // This could be from a TextBox input

string sql = "DELETE FROM department WHERE dno = @dno";
// ^^^^ placeholder

using (var conn = new MySqlConnection(connString))
{
 conn.Open();
 var cmd = new MySqlCommand(sql, conn);

 // Bind the value to the placeholder
 cmd.Parameters.AddWithValue("@dno", dno);
 // ^^^^^^^^^^^^ ^^^^^^ ^^^
 // method name name C# variable

 int rows = cmd.ExecuteNonQuery();
 MessageBox.Show(rows + " row(s) deleted.");
}
```

Understanding cmd.Parameters

The Parameters collection is a key security component of MySqlCommand. Understanding its structure helps avoid common mistakes.

```
// Retrieve the value that was sent to the database
string fname = cmd.Parameters["@Fname"].Value.ToString();
// ^^^^^^^^ ^^^^^ ^^^^^^^^^
// key getter convert object -> string

// Why not just use the original variable?
// Using .Parameters confirms what was ACTUALLY sent to the database
server,
// not just what was in your local variable before the call.
```

Part of Expression	Technical Name	What It Does
cmd.Parameters	MySqlParameterCollection	A list that holds all parameters for this SQL command.
["@Name"]	Indexer	Finds the specific parameter named @Name in the collection.
.Value	Property	The actual data stored inside the parameter (e.g., "John" or 42).
.ToString()	Method	Converts the generic object to a string for display.

?

When you create a parameter, C# creates an object called a **MySqlParameter**. This object is a container that holds **multiple pieces of information** about the parameter, not just the data itself:

- **The Name** — the placeholder label, e.g. @Name
- **The DbType** — the SQL data type, e.g. VarChar, Int32
- **The Size** — maximum length of the data, e.g. 50 characters
- **The Value** — the actual data being sent to the database (e.g. "John" or 42)

If you just type cmd.Parameters["@Name"], you are referring to the **entire MySqlParameter object** — not just the data inside it. To reach the actual text or number stored within, you must specifically ask for .Value:

The .Value property is typed as object because a parameter could hold any data type (string, int, date, binary). You must convert it explicitly.

User Interaction: MessageBox

MessageBox.Show() is the standard way to communicate results to the user in a Windows Forms application. It belongs to the System.Windows.Forms namespace.

Full Syntax

```
MessageBox.Show(message, title, MessageBoxButtons.XXX, MessageBoxIcon.XXX);
```

MessageBoxButtons Options

Enum Value	Buttons Displayed	Typical Use Case

MessageBoxButtons.OK	OK	Informational alerts with no decision required.
MessageBoxButtons.OKCancel	OK, Cancel	Confirm an action that can be aborted.
MessageBoxButtons.YesNo	Yes, No	Confirm a destructive action (e.g., delete).
MessageBoxButtons.YesNoCancel	Yes, No, Cancel	Three-way decision before exiting.

MessageBoxIcon Options

Enum Value	Icon	Use Case
MessageBoxIcon.Information	Blue ⓘ (info)	Success messages, general information.
MessageBoxIcon.Warning	Yellow ⚠ (triangle)	Caution before proceeding.

Enum Value	Icon	Use Case
MessageBoxIcon.Error	Red ✗ (cross)	Failed operations, exceptions.
MessageBoxIcon.Question	Blue ? (question)	Asking the user to make a decision.

String Conversion Rules

MessageBox.Show() only accepts strings. Passing any other type causes a compile error.

```
int dno = 6;

// ❌ FAILS — dno is an int, MessageBox needs a string
MessageBox.Show(dno);

// ✅ WORKS — explicit conversion
MessageBox.Show(dno.ToString());

// ✅ WORKS — string concatenation auto-calls ToString()
MessageBox.Show("Department ID: " + dno);

// ✅ WORKS — string interpolation (most readable)
MessageBox.Show($"Department ID: {dno}");
```

Lab Task — Secure Employee Insert

⁇ Task Brief

Write a C# Windows Forms method that securely inserts a new employee record (first name and last name) into the employee table using a parameterized MySQL query. After a successful insert, display a MessageBox showing the names that were inserted. If the operation fails, display the error message.

Execution Steps

1. Open the database connection inside a using block.
2. Define the parameterized INSERT SQL query with @Fname and @Lname placeholders.
3. Create the MySqlCommand object.
4. Bind the first name and last name values using AddWithValue().
5. Execute the command and capture the number of affected rows.
6. Retrieve the inserted values from cmd.Parameters[].Value.
7. Display success or failure message using the ternary operator.

Code Concepts Explained

Code Element	What It Is	Why It Is Used
@Fname, @Lname	SQL placeholders (parameters)	Prevent SQL injection by separating SQL from data.
AddWithValue()	Method of MySqlParameterCollection	Binds a C# value to a SQL placeholder safely.
cmd.Parameters["@Fname"].Value	Indexer + property access	Retrieves the exact value sent to the database server.
.ToString()	Object method	Converts the generic object type to a string for display.
rows > 0 ? ... : ...	Ternary (conditional) operator	Concise if-else for choosing between two values.
$"text {variable}"	String interpolation	Embeds variable values directly inside a string literal.

⬛⬛ Naming convention for the controls

Textbox: Txt

List: Lst

Button: Btn

Label: Lbl

Combo box: Cbo

Image: Img

Grid: Grd

We will use the PascalCase naming convention where the first letter of each compound word in a variable is capitalized.